

ADR-0005 — OIDC via Keycloak; ownership enforced in the application layer

ACCEPTED Date: 2026-09-07 · Deciders: Nicolà Widmer

Context

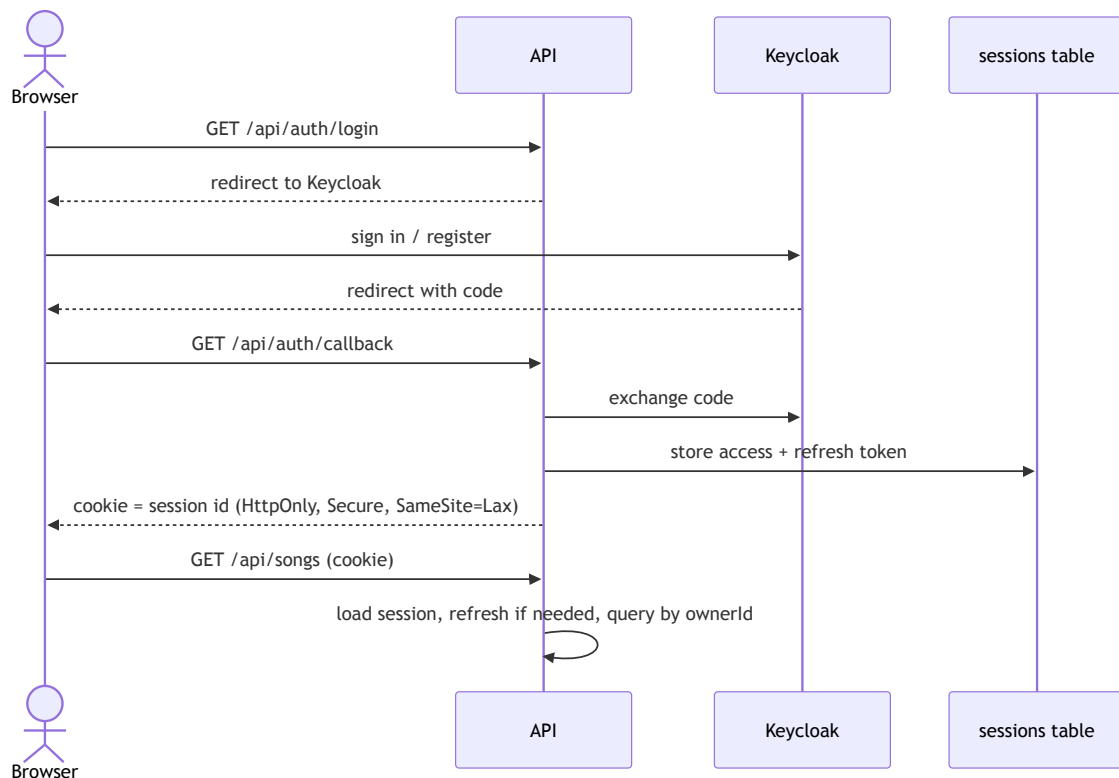
Users sign in (AUTH-1, AUTH-2), and all data is private to its owner. Running our own password store means building reset, lockout and hashing ourselves. <audio> and cannot send an Authorization header.

Options

1. **Keycloak (OIDC); API is a confidential client with a session cookie (BFF); owner check in every query**
2. Own email/password store with session cookies
3. Own JWTs, no identity provider
4. Owner check only in HTTP middleware

Decision

Option 1.



- Keycloak runs in Compose with its own database.
- The app keeps a thin `users` table (`id = Keycloak sub`) so foreign keys work.
- Every repository query is scoped by `ownerId` — other users' data is 404.
- Logout deletes the session and ends the Keycloak session.
- `AUTH_ENABLED=false` uses one fixed local user.

Consequences

- Good: no password code in this project; registration comes free.
- Good: no token in the browser; media requests work with the cookie.
- Good: any OIDC provider fits.
- Bad: an extra service + database; slow first boot.
- Bad: the API must store and refresh tokens and needs a client secret.

- Bad: local `users` rows can go stale if a user is deleted in Keycloak.
- Bad: owner scoping relies on convention and tests.